

Networking Project



2 Contributors

System Architecture

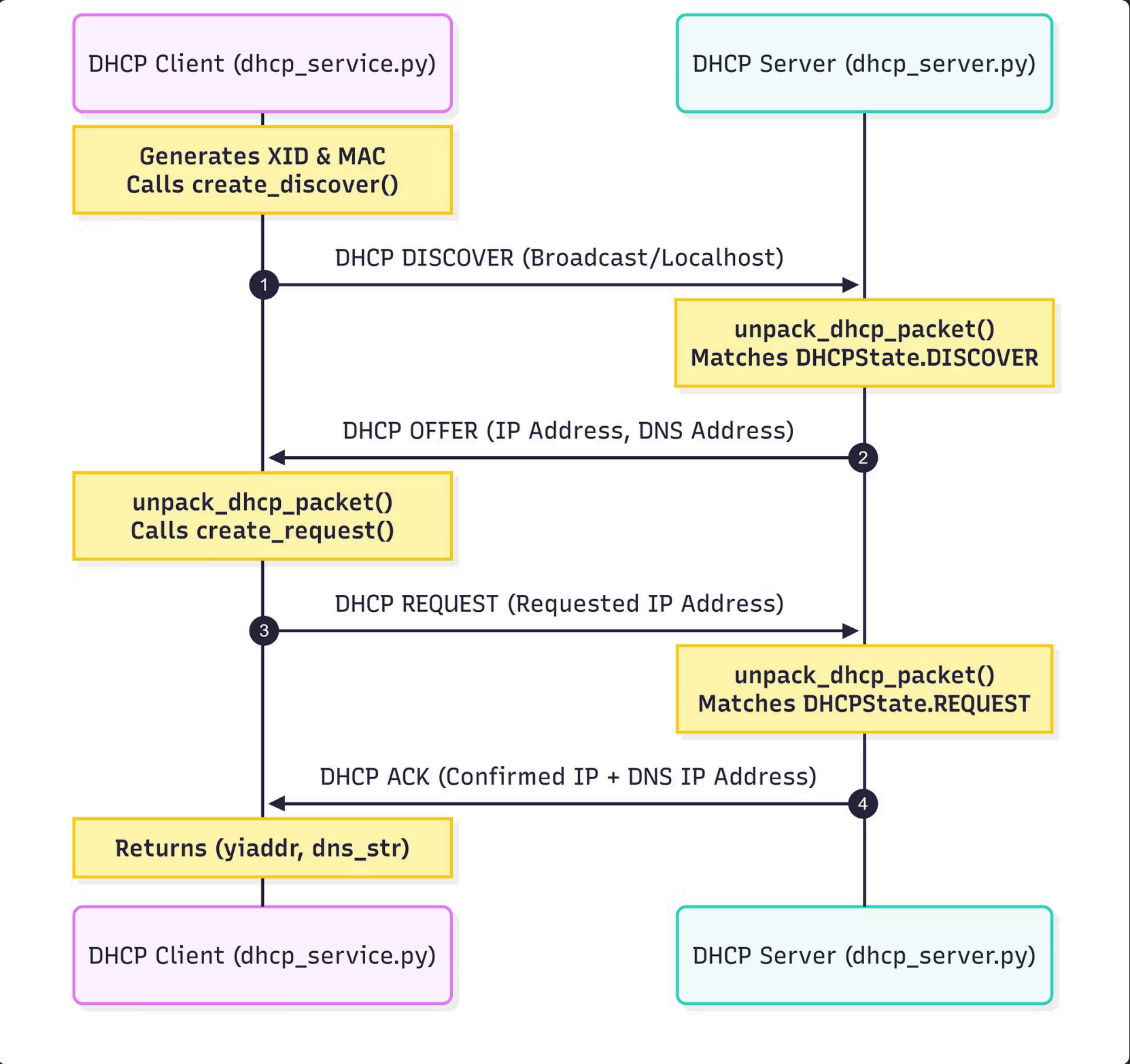
The project is built as a small end-to-end application-layer network stack composed of DHCP, DNS, FTP, and a custom RUDP transport. The goal is to simulate how a client joins a network, discovers infrastructure services, resolves a service name, and finally downloads a file.

Two Implementation Paths

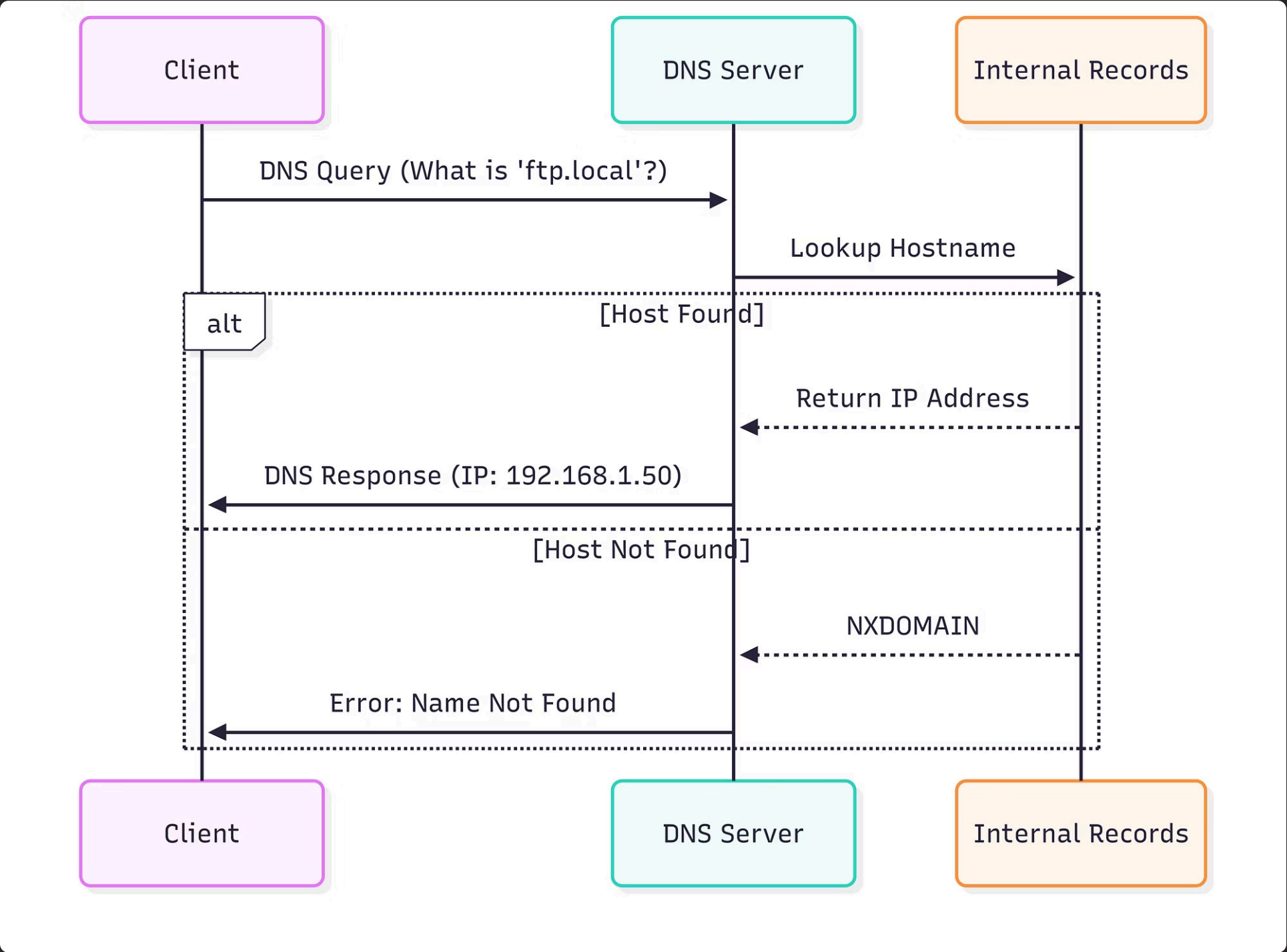
- **RFC-inspired path:** DHCP and DNS encoded in binary structures similar to real protocol formats.
- **JSON-based path:** Same logical flow with human-readable JSON messages for easier debugging and Wireshark inspection.



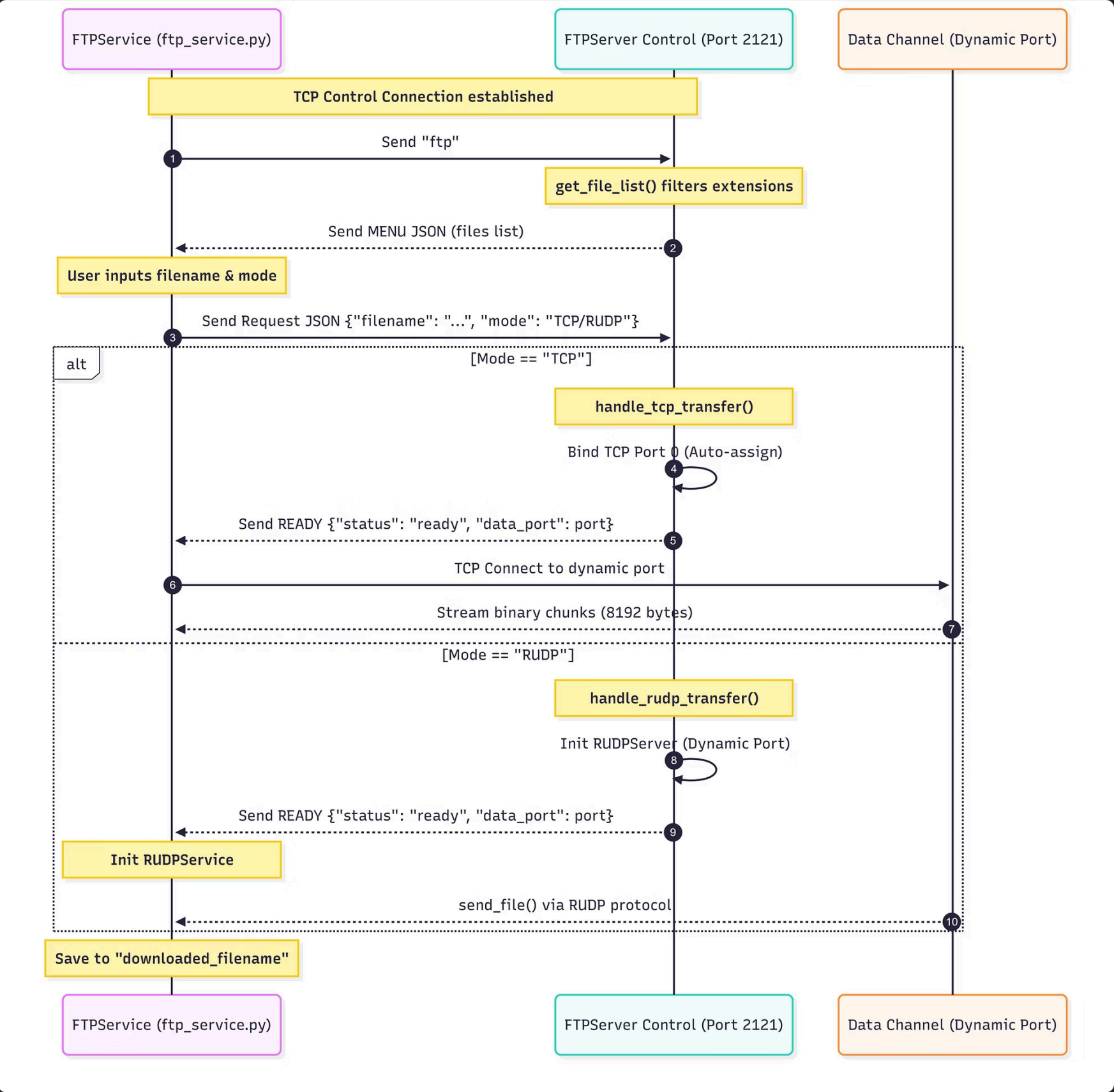
DHCP Workflow



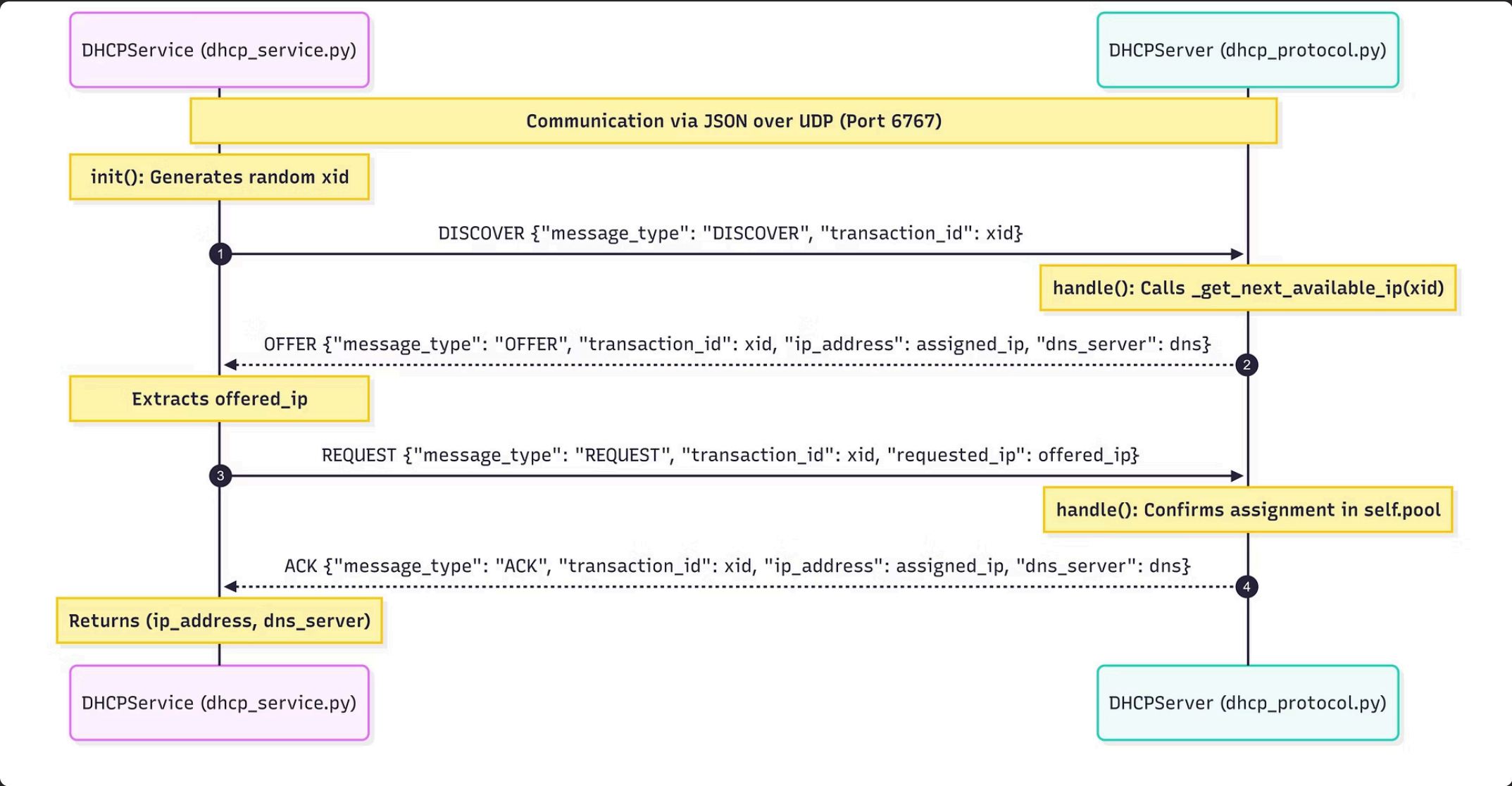
DNS Workflow



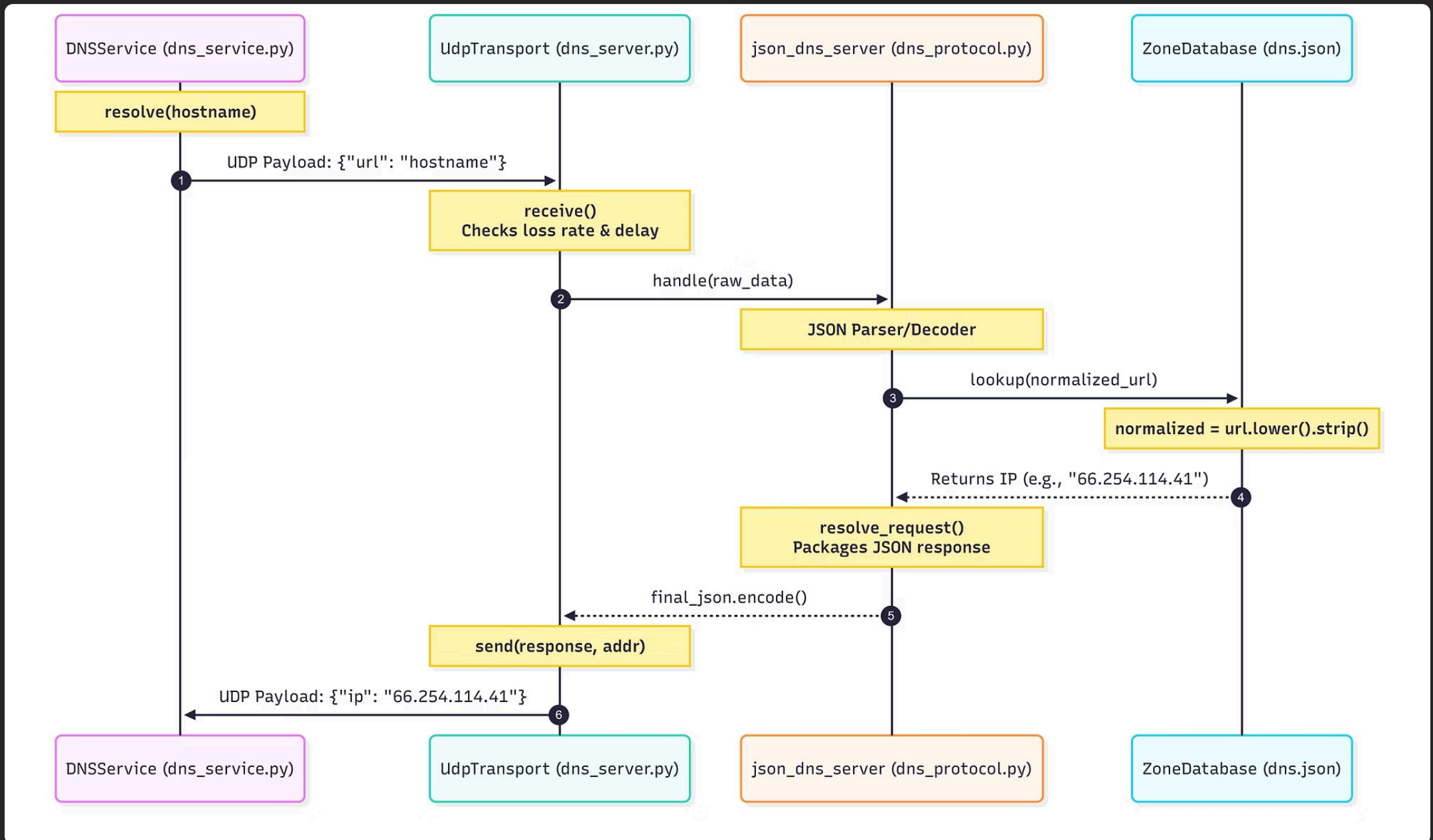
FTP Workflow



DHCP JSON Workflow



DNS JSON Workflow



DNS

The DNS module serves as a streamlined implementation of an authoritative DNS server and a client resolver, utilizing the UDP protocol. This module provides a practical demonstration of internal DNS mechanics, covering manual (binary) packet parsing, zone database queries, and the construction of binary responses.

Key Components:

ZoneDatabase: Manages and retrieves DNS records from JSON-formatted files.

Protocol Handling: Includes a parser and response builder to handle binary packet structures.

UDP Transport: Facilitates efficient network communication between the server and client.

DNSServer & Service: The server core processes incoming requests, while the client service handles outbound queries and IP extraction.

Operating on UDP port 8053, the system functions as a lightweight authoritative server, resolving queries against zones defined in the `dns.json` configuration.

DNS_binary/dns_protocol.py

This code implements a simple authoritative DNS server that receives DNS requests, parses them, looks up domains in a local database, builds proper DNS response packets, and sends them back.

Key Components:

DNSHeader: Parses the first 12 bytes of a DNS packet, extracting transaction ID, flags, and record counts.

ZoneDatabase: Acts as a mini DNS database, loading records from `dns.json` and mapping domains to IPs.

DNSQuestion: Parses the Question section of DNS requests, decoding domain names and validating packet structure.

DNSResponseBuilder: Constructs response packets including headers, question sections, A records, and SOA records for error handling.

Workflow: DNS request → parse packet → lookup domain → build response → return packet. It supports A records, SOA authority responses, NXDOMAIN/NODATA handling, and basic DNS packet validation.



DNS_binary/dns_server.py

This file implements the actual DNS server runtime, handling UDP socket operations, receiving DNS requests, and sending back DNS responses. While the previous file handled DNS packet parsing and building, this file manages network communication and request processing.

Key Components:

UdpTransport

Handles low-level UDP networking by opening a UDP socket, binding to 127.0.0.1:8053, receiving incoming packets, and sending responses. It also supports simulating network problems like packet loss and artificial delays.

DNSServer

The core DNS logic that connects the database, packet parser, and response builder. It validates packets, parses DNS requests, checks zone authority, looks up domains in the local database, and handles different query types (A records, NXDOMAIN, NODATA).

Request Flow: Client sends DNS query → server receives packet → validates and parses request → checks zone authority → looks up domain in dns.json → builds appropriate DNS response (A record, NXDOMAIN with SOA, or NODATA) → sends response back to client.

Error Handling: Returns SERVFAIL (RCODE = 2) for processing errors, REFUSED (RCODE = 5) for unauthorized zones, NXDOMAIN (RCODE = 3) for non-existent domains, and NODATA for unsupported record types.

Combined with the previous dns_protocol.py file, this creates a fully functional authoritative DNS server with UDP transport, request handling, database lookup, and proper error responses.

DNS_binary/dns_service.py

This is a simple DNS client that sends DNS queries to the DNS server and returns the IP address of a hostname. The flow is: hostname → send DNS query → receive DNS response → extract IP.

Key Components:

DNSService: A DNS resolver class that communicates with the DNS server on port 8053. It uses UDP for communication with a buffer size of 65535 bytes and a 5-second timeout for responses.

resolve(hostname): The main function that resolves a domain to an IP address. It creates a UDP socket, builds a proper DNS query packet, sends it to the DNS server, receives the response, and extracts the IP address from the last 4 bytes of the A record response.

_build_query(): Constructs a DNS request packet with the proper structure. The header includes transaction ID, flags (standard query), and record counts (1 question, 0 answers). The question section encodes the hostname in DNS wire format (e.g., www.example.com becomes \x03www\x07example\x03com\x00) and specifies TYPE A (record type 1) and CLASS IN (class 1).

Architecture: DNS Client → UDP → DNS Server → Zone Database → DNS Response. This implements a minimal DNS resolver that demonstrates how real DNS clients work by building queries, sending them via UDP, receiving responses, and extracting IP addresses from A records.



Database

The DNS zone database stores domain-to-IP mappings in JSON format. This database is queried by the DNS server to resolve hostnames to their corresponding IP addresses.

```
{  
  "example.com": "127.0.0.1",  
  "www.example.com": "127.0.0.2",  
  "mail.example.com": "127.0.0.3",  
  "ftp.project_gay.com": "127.0.0.4",  
  "ns1.example.com": "127.0.0.5",  
  "ariel.example.com": "127.0.0.6",  
  "google.com": "142.250.75.110",  
  "www.google.com": "142.250.75.111",  
  "ftp.google.com": "142.250.75.112"  
}
```


DNS_JSON

In addition to the binary DNS implementation, the project includes a simplified DNS system using JSON messages. This version demonstrates DNS resolution logic without the complexity of binary protocol parsing.

Instead of parsing raw DNS packets, the system exchanges JSON messages over UDP. This approach makes the protocol easier to test and debug while still demonstrating core DNS concepts.

Key Features:

- UDP networking: for message transport
- DNS-style name resolution: with JSON payloads
- Database-backed domain lookup: using dns.json
- Request/response protocol design: with human-readable messages

Main Components:

JSON DNS Server:

Processes incoming DNS requests in JSON format and returns results as JSON responses. Queries the zone database to resolve domain names to IP addresses.

Zone Database (dns.json):

Stores domain name to IP address mappings in JSON format, enabling quick lookups for both local and external domains.

Advantages Over Binary DNS:

- Human-readable message format for easier debugging
- Simplified packet structure without binary encoding/decoding
- Better compatibility with Wireshark inspection
- Faster development and testing cycle
- Same logical DNS resolution flow as the RFC-compliant binary version

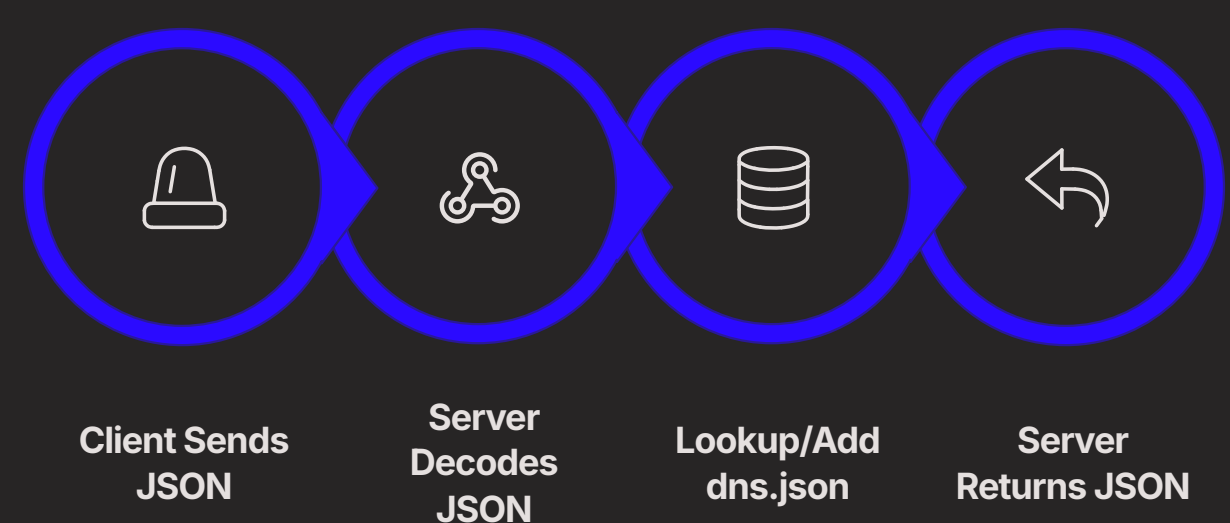
This JSON-based approach provides an excellent learning tool for understanding DNS mechanics while maintaining simplicity and clarity.



DNS_JSON/dns_protocol.py

This file implements a simplified JSON-based DNS protocol that exchanges human-readable JSON messages over UDP instead of using binary DNS packets.

Request/Response Flow:



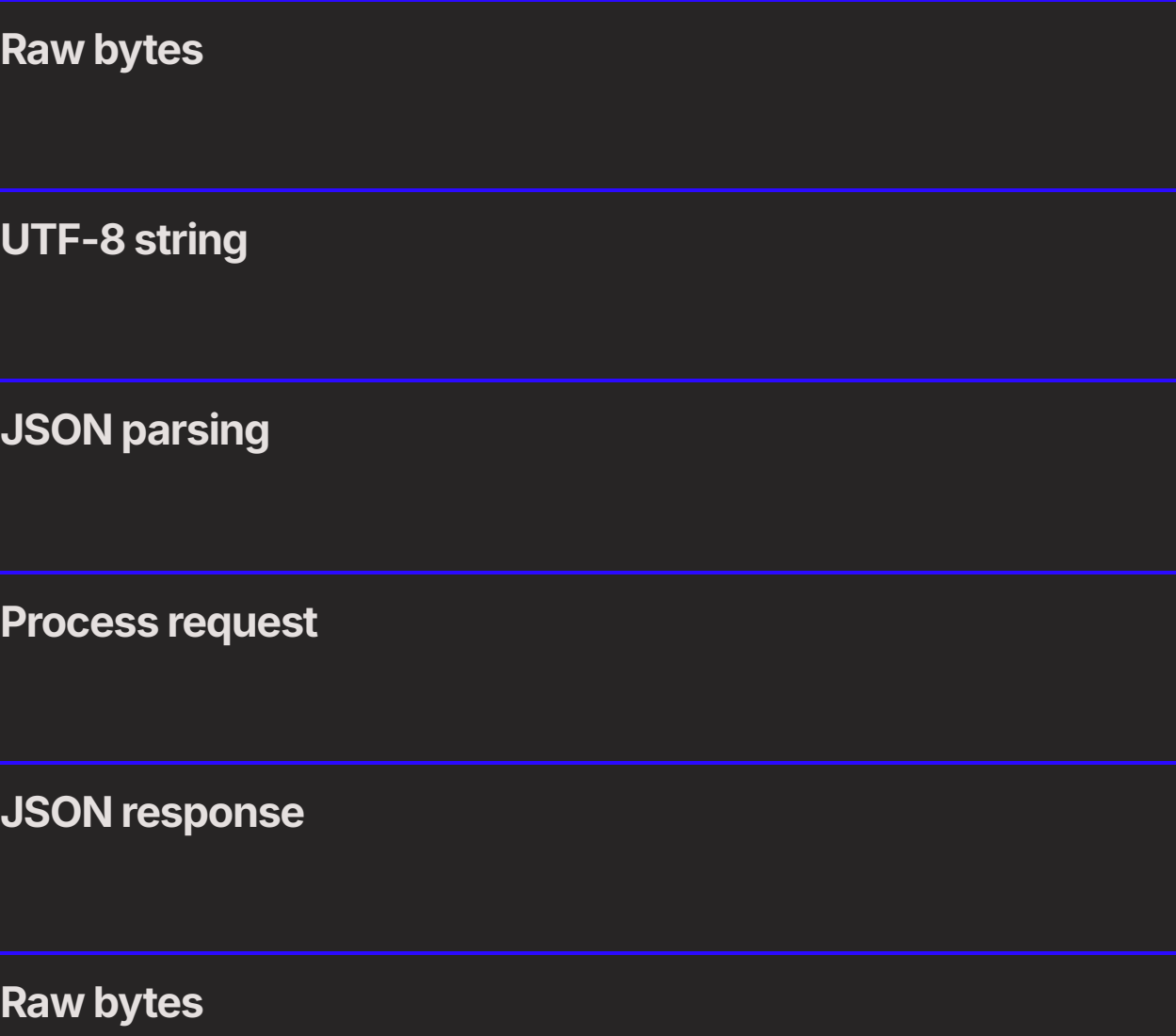
Example Messages:

```
Lookup request: {"url":"example.com"}
Add record request: {"url":"example.com","ip":"1.2.3.4"}
Response: {"ip":"127.0.0.1"}
```

Key Components:

	ZoneDatabase Manages the local DNS database by loading records from dns.json, saving records back to file, adding new records, and checking whether domains exist. Provides lookup functionality to retrieve IP addresses by domain name.
	add_record(url, ip) Adds a new domain-to-IP mapping with validation. Uses socket.inet_aton(ip) to ensure the IP is a valid IPv4 address before storing.
	resolve_request(database, request_dict) Handles two main operations: - Add Record: If request contains both url and ip, validates input, adds the record to the database, and returns the saved IP - Lookup Record: If request contains only url, checks if it exists and returns its IP, or returns {"ip": None} if not found
	json_dns_server The server handler that processes raw bytes by decoding UTF-8, parsing JSON, calling resolve_request(), and converting results back to JSON bytes. Handles errors by returning {"error":"Server error"}.

Workflow:

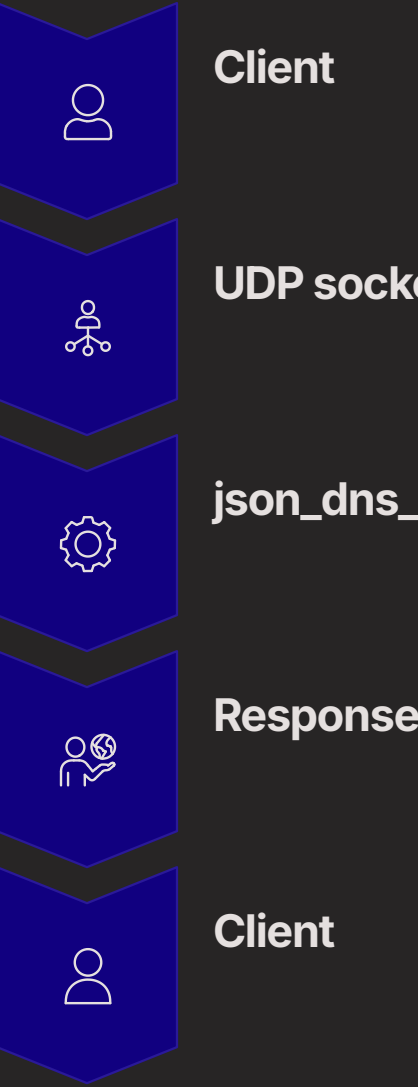


This JSON-based approach provides a simplified, debuggable alternative to binary DNS that demonstrates core DNS concepts like domain lookup and record management without the complexity of packet parsing.

DNS_JSON/dns_server.py

This file implements the UDP runtime for the JSON-based DNS server. It opens a UDP socket, waits for client messages, passes received bytes to the JSON handler, and sends responses back.

Request Flow:



Key Components:

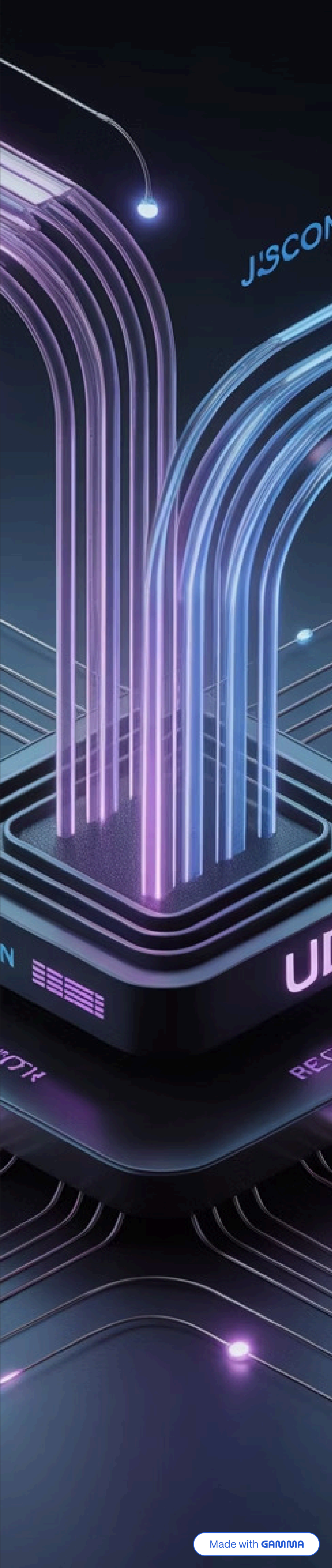
- UdpTransport:** Handles the network layer by opening a UDP socket, receiving packets, sending packets, and closing the socket. It also supports simulating network problems like packet loss and artificial delays for testing purposes.
- initialize():** Creates and binds a UDP socket using socket.AF_INET (IPv4) and socket.SOCK_DGRAM (UDP protocol).
- receive():** Waits for incoming data from a client using recvfrom(). Returns the raw request bytes and the sender's address.
- send():** Sends response bytes back to the client at the same address that sent the request.
- run_dns_server(host, port):** The main server loop that creates the UdpTransport, initializes the ZoneDatabase, instantiates the json_dns_server handler, and continuously loops to receive and process requests. Each iteration receives a request, calls json_server.handle(data), and sends the response back.
- serve():** A helper function that starts the server on 127.0.0.1:9000.

Architecture:

This file handles only communication and the server loop. The actual DNS logic (domain lookup and record management) is contained in the ZoneDatabase and json_dns_server components. This separation of concerns keeps the transport layer independent from the application logic.

Summary:

This file is the UDP runtime for the JSON-based DNS server, listening on port 9000, receiving JSON requests, processing them through the handler, and sending JSON responses back to clients.



FTP

The FTP component implements a simplified, FTP-inspired file transfer system that allows clients to browse and download media files from a server. It uses a two-channel architecture similar to classic FTP: a persistent TCP control channel for commands and negotiation, and a separate temporary data channel for actual file transfer.

Key Feature - Hybrid Data Channel:

The data channel supports two different modes:

- TCP: Uses the operating system's built-in reliability
- RUDP: A custom Reliable UDP protocol implemented as part of the project

This hybrid design keeps the control plane on TCP while allowing payload transfer via either standard TCP or custom UDP-based reliability.

Main Components:

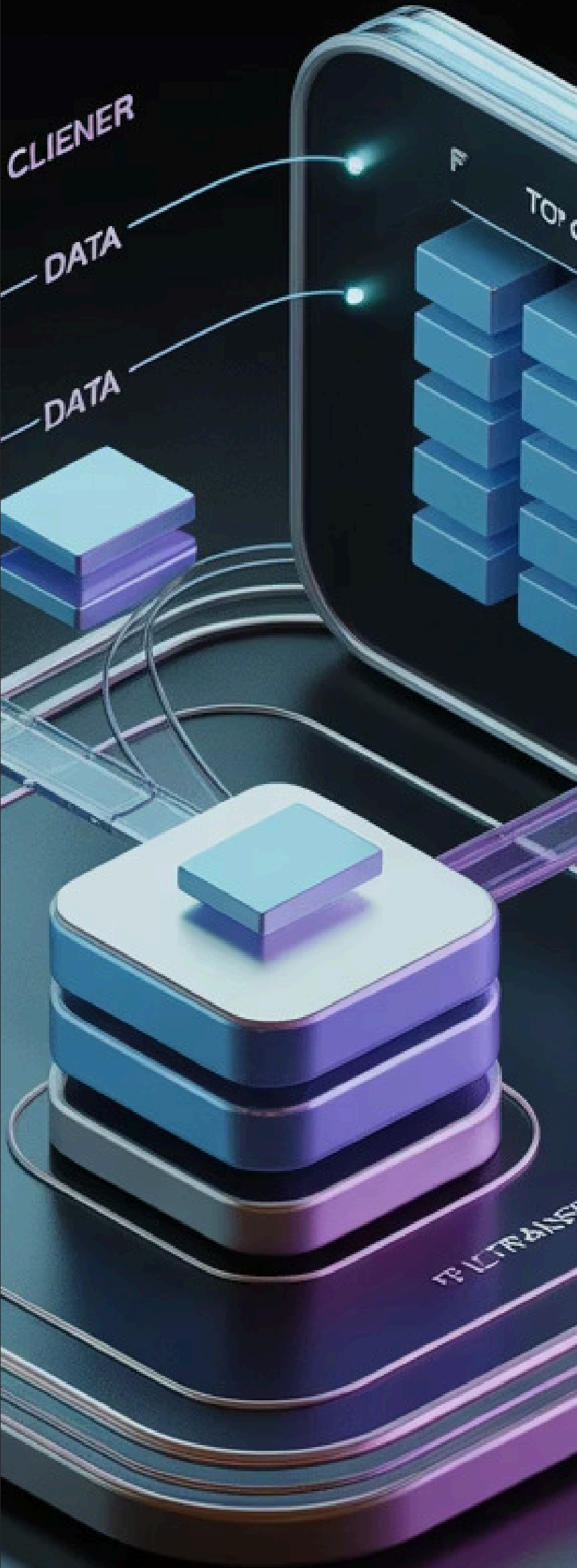
ftp_server.py: Server-side FTP logic that manages control connections and file transfers	ftp_service.py: Client-side FTP logic that initiates connections and requests files
rudp_server.py: Sends files using the custom RUDP protocol	rudp_service.py: Receives files using the custom RUDP protocol
rudp_protocol.py: Defines the RUDP packet format and control flags	

Complete Workflow:

1. Client receives IP address from DHCP
2. Client resolves FTP server hostname using DNS
3. Client opens TCP control connection to FTP server on port 2121
4. Client requests the file menu from the server
5. Client selects a file and chooses TCP or RUDP for transfer
6. Server creates a temporary data port and sends the port number to the client
7. Client connects to the temporary port and downloads the file

Architecture Principle:

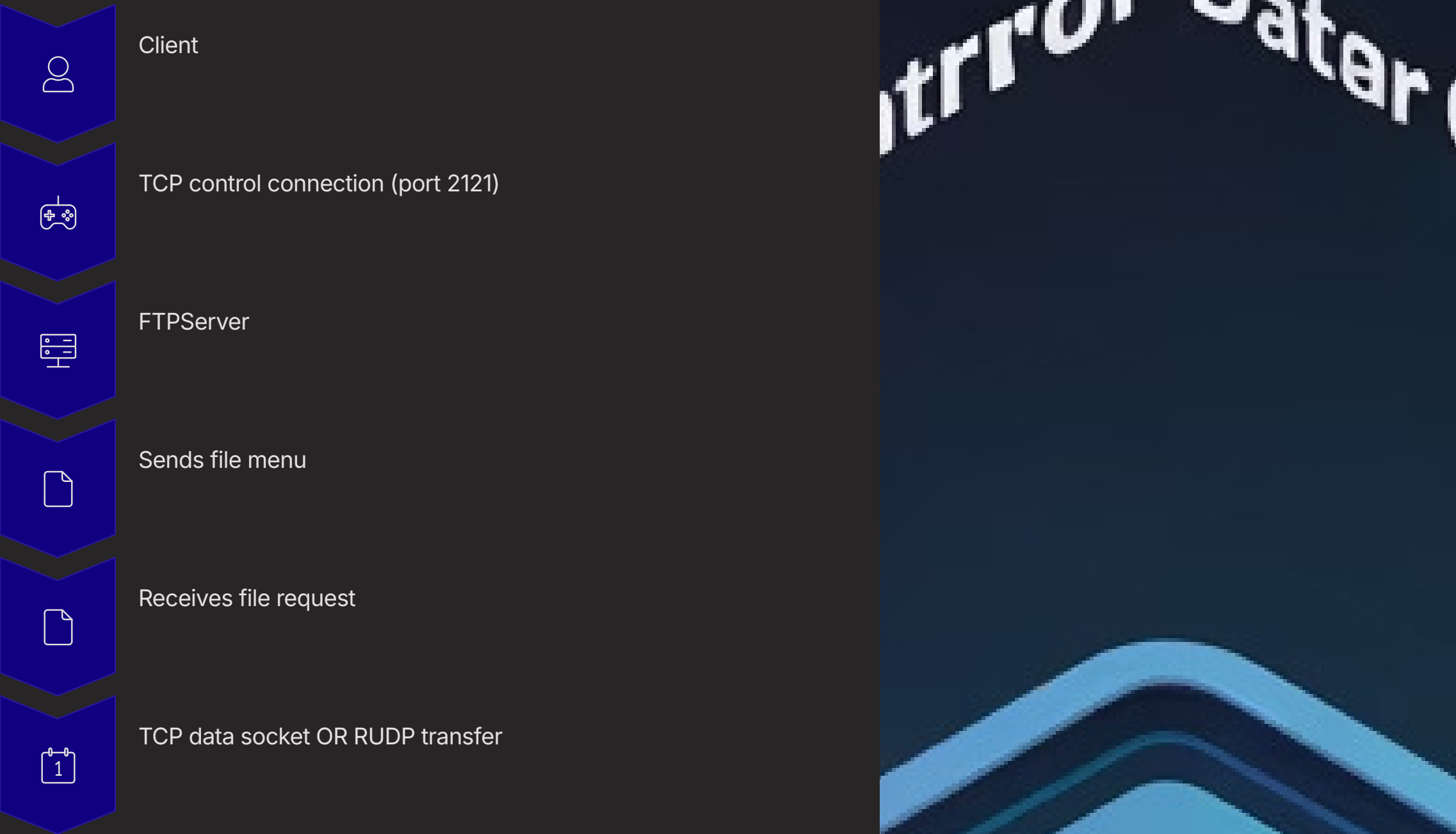
Control traffic and data traffic are separated. The control connection remains open for the entire session, while the data connection exists only for the duration of a single file transfer. This separation allows efficient resource management and supports multiple concurrent transfers.



FTP/ftp_server.py

This is an FTP-like server that supports two file transfer modes: TCP and RUDP (Reliable UDP). It follows the classic FTP architecture with two separate sockets—one for control commands and another for data transfer.

Architecture:



Key Components:

	FTPServer The main class that manages the server. It runs the control channel on port 2121 and serves files from the "server_files" directory. Supports multiple concurrent clients using threading.
	start_server() Creates the control socket using TCP (AF_INET, SOCK_STREAM), listens for incoming connections, and spawns a new thread for each client to handle their session independently.
	handle_client_session() Manages one client session by waiting for commands. Clients can send "ftp" to request the file menu or "quit" to disconnect. The server responds with a JSON menu containing available media files (.mp3, .jpg, .jpeg, .gif, .mp4).
	File Request Handling: When a client requests a file, they send a JSON message specifying the filename and transfer mode (TCP or RUDP): <code>{"filename":"song.mp3","mode":"TCP"}</code> or <code>{"filename":"song.mp3","mode":"RUDP"}</code>
	TCP File Transfer (handle_tcp_transfer()) Creates a new data socket on a dynamically chosen port, sends the port number to the client, accepts the client connection, and streams the file in 8KB chunks using sendall().
	RUDP File Transfer (handle_rudp_transfer()) Uses the custom RUDPServer protocol instead of TCP. Creates a RUDP handler, chooses a UDP port, sends the port to the client, and transfers the file via the reliable UDP implementation.
	get_file_list() Returns up to 10 media files from the server directory, filtering for allowed formats only.

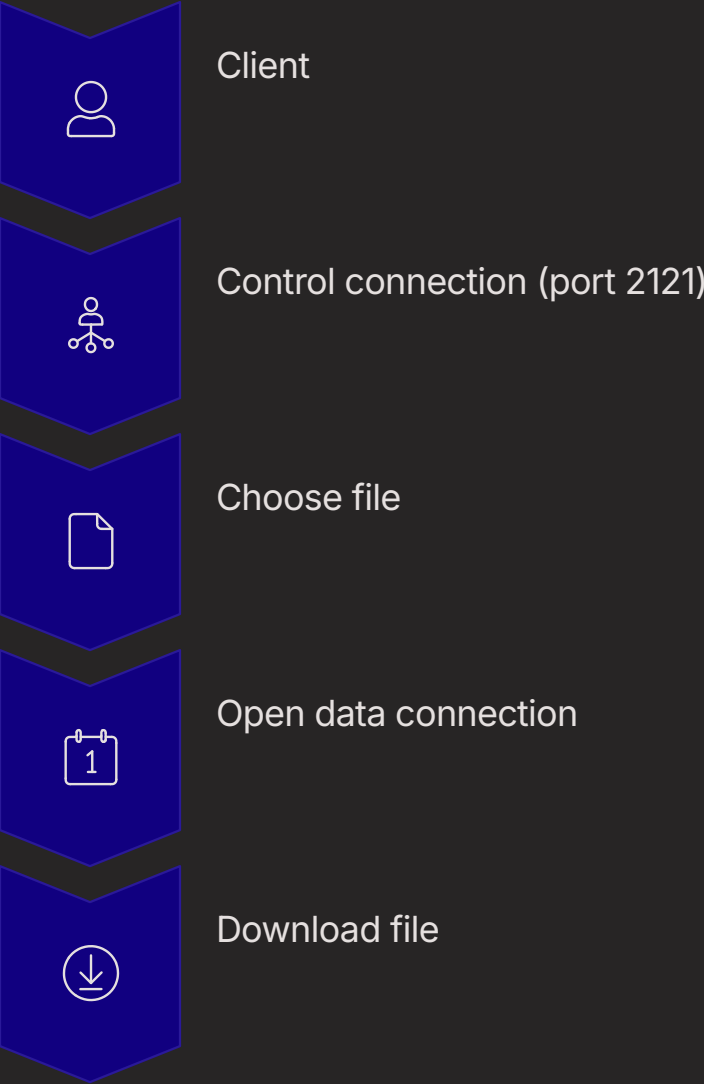
Summary:

This server demonstrates socket programming, multithreading, control vs. data channel separation, file streaming, and integration with a custom reliable protocol (RUDP). The two-channel design allows the control connection to remain open while temporary data connections handle individual file transfers.

FTP/ftp_service.py

This is the FTP client that connects to the FTP server, requests a file list, allows the user to choose a file, and downloads it using either TCP or RUDP.

Request Flow:



Key Components:

FTPService:

The main class implementing client-side FTP logic. Uses port 2121 for the control connection and supports a maximum message buffer size of 65535 bytes.

_receive_tcp():

Handles file downloads via TCP. Connects to the data port provided by the server, receives file chunks in a loop, and writes them to a local file named "downloaded_[filename]".

_receive_rudp():

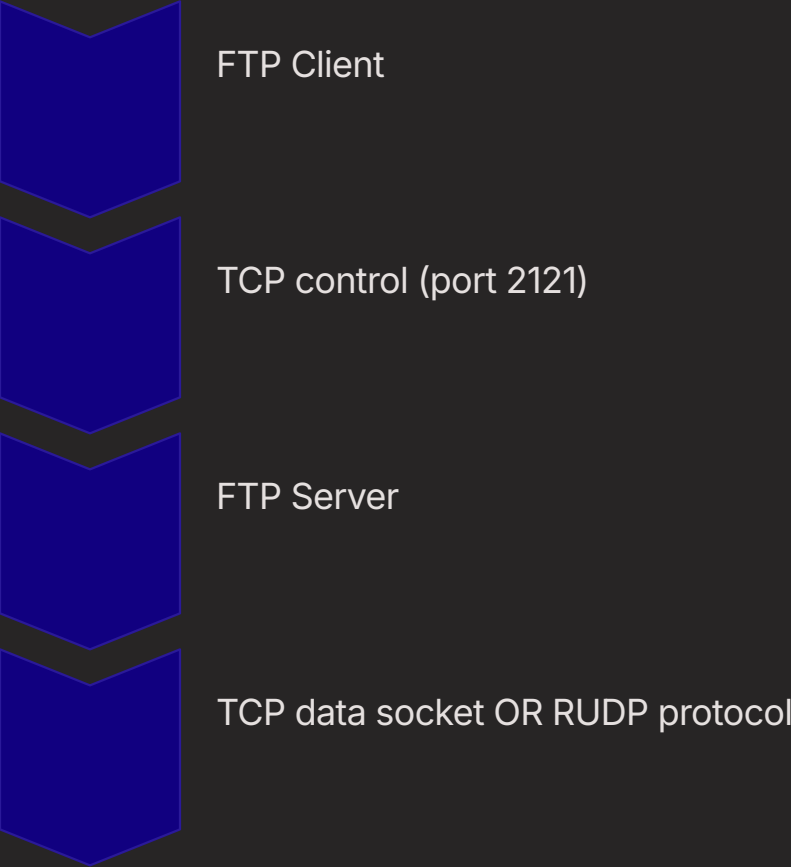
Handles file downloads using the custom RUDP protocol. Creates a RUDP client service, receives the file through the reliable UDP implementation, and saves the bytes to disk.

run_file_transfer():

The main client loop that orchestrates the entire file transfer process:

1. Connects to the server on port 2121 (control channel)
2. Prompts user for command ("ftp" to request file list or "quit" to disconnect)
3. If "ftp": Server sends JSON menu with available files
4. User selects file number and transfer mode (TCP or RUDP)
5. Client sends request: {"filename":"song.mp3","mode":"TCP"}
6. Server responds with data port: {"status":"ready","mode":"TCP","data_port":45000}
7. Client downloads file using selected mode

Architecture:



This mimics real FTP architecture by separating control and data connections. The control connection remains open for the entire session, while the data connection is created temporarily for each file transfer.

Summary:

This command-line FTP client connects to the server, retrieves the file menu, lets users choose a file and transfer mode, and downloads files using either TCP or custom RUDP protocol.

RUDP/rudp_protocol.py

This file defines the RUDP (Reliable UDP) packet structure and parsing logic. Since UDP itself is unreliable, this protocol adds sequence numbers, acknowledgements, and connection flags to ensure reliable delivery.

Packet Structure:

[SEQUENCE_NUMBER][TOTAL_PACKETS][FLAGS][PADDING] | PAYLOAD

Header Format (12 bytes total):

- Sequence Number (4 bytes): Tracks packet order
- Total Packets (4 bytes): Total number of packets in transfer
- Flags (1 byte): Control flags (DATA, FIN, ACK, SYN)
- Padding (3 bytes): Alignment padding

Maximum Payload Size:

1450 bytes
Chosen to prevent fragmentation (Ethernet MTU ~1500 bytes minus IP and UDP headers)

Control Flags:

- DATA_FLAG (0x00): Normal file data packet
- FIN_FLAG (0x01): End of transfer signal
- ACK_FLAG (0x02): Acknowledgement packet
- SYN_FLAG (0x04): Connection start signal

These flags mimic TCP control flags for reliable communication.

Key Functions:

create_packet(seq_num, total_packets, data, flags):

Packs the header using struct.pack(), attaches the payload, and returns a binary packet ready to send.

parse_packet(packet):

Validates packet size, extracts the 12-byte header, extracts the payload data, decodes header fields using struct.unpack(), and returns sequence number, total packets, flags, and data.

create_ack(seq_num):

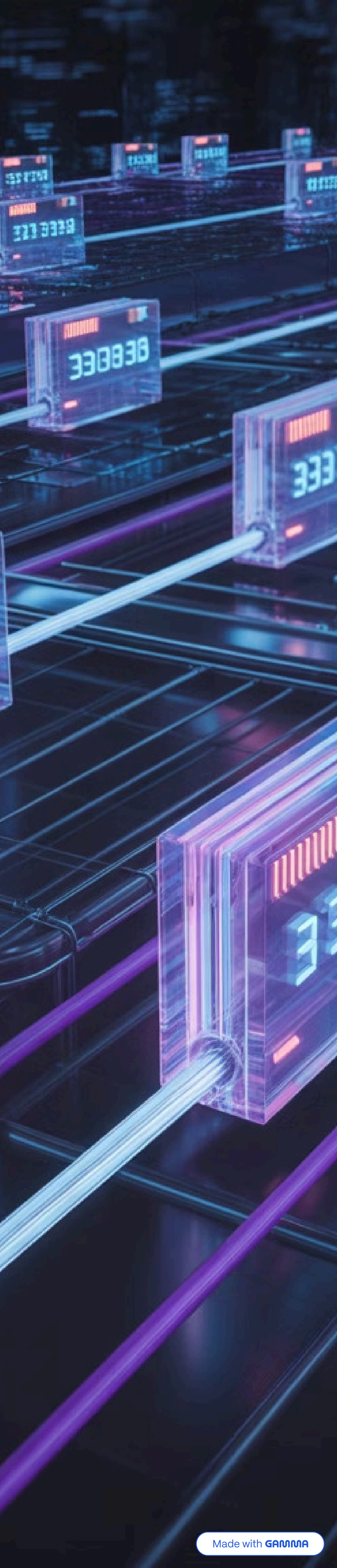
Creates an acknowledgement packet with the ACK flag set, no payload, and the sequence number of the packet being acknowledged.

Protocol Flow:



Summary:

This class implements the RUDP packet protocol layer, acting as a custom TCP-like header for UDP. It provides sequence tracking, acknowledgement support, and connection control flags to transform unreliable UDP into a reliable transport mechanism for file transfers.



RUDP/rudp_server.py

This code implements a Reliable UDP (RUDP) Server using a Sliding Window protocol (specifically a variation of Go-Back-N). Since standard UDP is "fire and forget," this class adds a custom reliability layer to ensure file data actually reaches the client in the correct order.

Core Components and Workflow:

Initialization & Handshake:

It initializes a standard UDP socket (SOCK_DGRAM) but doesn't start sending immediately. It waits for a SYN (Synchronize) flag from a client, which acts as a trigger to begin the transfer.

Packetization:

The file_bytes are sliced into chunks based on a MAX_PAYLOAD_SIZE. Each chunk is wrapped in a custom RUDPProtocol header containing sequence numbers and flags.

The Binary ACK Listener (Threading):

A background thread (listen_for_acks) constantly monitors the socket for incoming packets. It parses these to find ACK (Acknowledgment) flags. When a client confirms it received packet N, the server updates self.last_ack.

The Sliding Window Mechanism:

Instead of sending one packet and waiting, it sends a "window" of packets (e.g., 10 at a time). Upper Bound: It calculates how many packets it can send ahead of the last acknowledged packet: Window Upper Bound = last_ack + window_size. Retransmission: If the last_ack stops moving forward, the code resets next_to_send to last_ack + 1, effectively re-sending the missing data.

Teardown (FIN):

Once all data is acknowledged, it sends a FIN (Finish) packet multiple times (to account for potential loss) to tell the client the stream is over, then closes the socket.

RUDP/rudp_service.py

building Reliability on top of UDP. Since standard UDP is "fire and forget," this RUDPService class implements a custom stop-and-wait or selective-repeat mechanism to ensure a file actually arrives intact.

Initialization (`__init__`):

The service initializes a standard UDP socket (`SOCK_DGRAM`). It binds to a local IP but lets the OS pick an ephemeral port (0). It also sets a timeout, which is critical because, unlike TCP, UDP won't naturally tell you if the other side vanished.

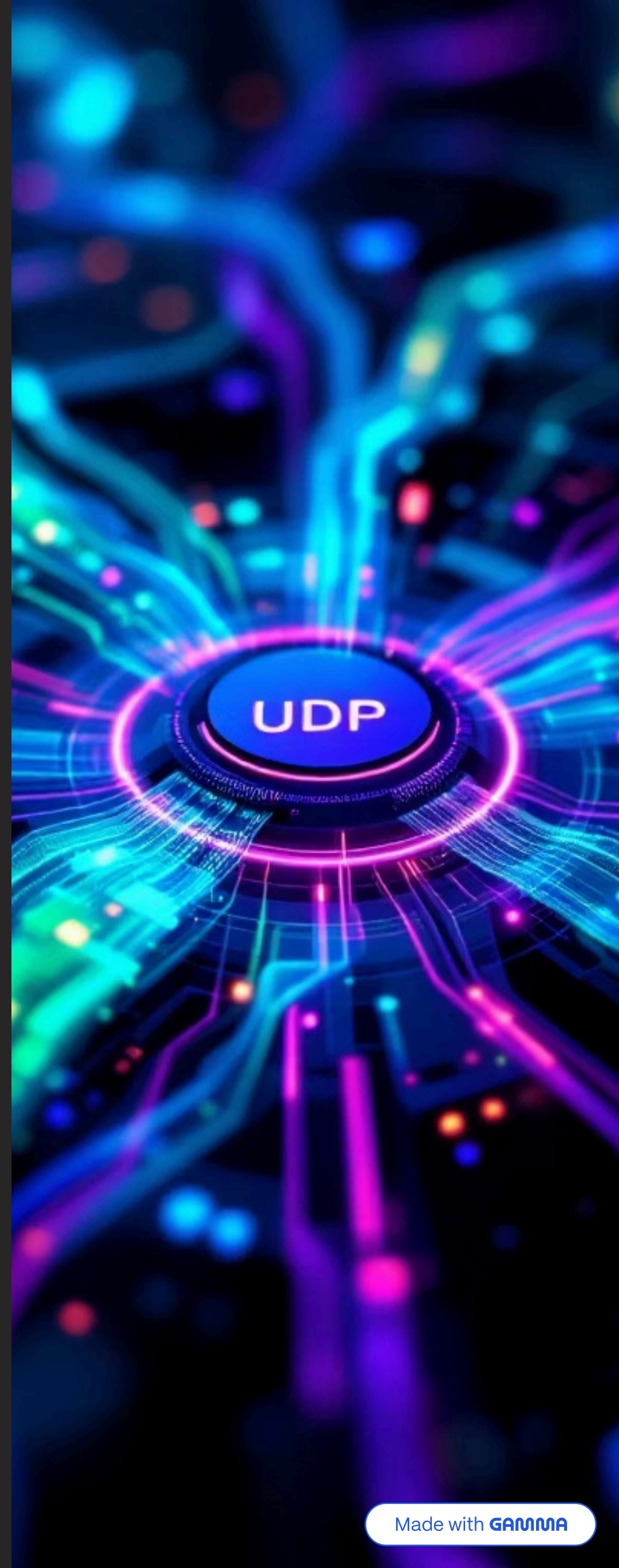
The Handshake & Reception (`receive_file`):

The process follows a three-step dance:

1. **The Trigger:** It sends a SYN flag packet to the server. This acts as a "Start" command to tell the server to begin streaming the file chunks.
2. **The Receive Loop:** It listens for incoming data chunks. For every chunk received, it sends an ACK (Acknowledgement) back to the server. This is the "Reliable" part of RUDP; if the server doesn't get that ACK, it will likely re-transmit that specific chunk. It uses a dictionary (`received_chunks`) to store data by sequence number, ensuring that even if packets arrive out of order, they can be organized later.
3. **The Termination:** It looks for a FIN flag. Once seen, the loop breaks, signaling that the server has no more data to send.

Reconstruction:

Once the loop finishes, it iterates through the keys of the dictionary to stitch the binary chunks back together in the correct order. Finally, it closes the socket and returns the complete byte buffer.



DHCP/dhcp_model.py

DHCPPacket Data Structure

The dataclass defines the standard fields found in a UDP-encapsulated DHCP packet.

Packet Anatomy:

It covers the fixed-length header fields (like xid for tracking transactions and chaddr for the client's hardware address) and includes an options dictionary for the variable-length fields (like Subnet Mask or Router IP).

Automation:

Using a dataclass here is smart—it automatically handles initialization (`__init__`) and makes it easy to serialize these fields into the binary format required to send them over a real network socket.

DHCPState (The DORA Process)

The DHCPState enum maps the human-readable steps of the DORA process to their specific byte values used in Option 53 (DHCP Message Type).

01

DISCOVER:

Client looking for servers.

02

OFFER:

Server responding with potential IP.

03

REQUEST:

Client asking for that specific IP.

04

ACK:

Server confirming the lease.

How it all fits together

In a networking script, you would use these to maintain the State Machine. For example, when your script receives a packet where the `options[53] == DHCPState.OFFER`, it knows to prepare a `DHCPPacket` with `op=1` (Request) to keep the handshake moving.



DHCP/dhcp_protocol.py

Binary Serialization (struct):

The core of the script uses Python's struct module to map high-level data (IPs, MAC addresses, IDs) into the specific binary format required by RFC 2131.

- pack_dhcp_packet:** Converts a DHCP packet object into a byte string. It includes the fixed-length header (44 bytes), 192 bytes of padding for legacy fields (sname/file), the Magic Cookie (0x63825363), and the variable-length options.
- unpack_dhcp_packet:** The inverse; it slices a raw byte stream to reconstruct a readable Python object.

The D.O.R.A. Factory:

The script provides helper functions to generate the four specific message types used when a client joins a network:

Function	DHCP Step	Description
create_discover	Discover	Client broadcasts to find available servers.
create_offer	Offer	Server proposes an IP (yiaddr) and DNS settings to the client.
create_request	Request	Client formally asks to lease the offered IP.
create_ack	Acknowledge	Server confirms the lease and provides final parameters.

Data Handling:

Network Byte Order: The code uses the ! prefix in struct.pack, ensuring "Big Endian" byte order, which is the standard for network protocols.

Configuration: It pulls the dns_ip from a local dhcp.json file, allowing the server-side responses (OFFER and ACK) to tell clients which DNS server to use.

Technical Observations:

- Option 53:** You'll notice p.options[53] is set in every function. This is the DHCP Message Type option, which is mandatory for the protocol to function.
- Magic Cookie:** Without that specific 4-byte sequence (0x63825363), a DHCP client or server will reject the packet as invalid.
- Memory Safety:** The chaddr (Client Hardware Address) is padded to 16 bytes using .ljust(16, b'\x00'), which is correct per the spec even though most MAC addresses are only 6 bytes.



DHCP/dhcp_server.py

1

Initialization & Pool Management:

The DHCPServer class defines a static IP pool. It takes your server's IP (e.g., 192.168.1.1) and generates a range of assignable addresses from .100 to .200.

2

The Socket Setup (Networking Layer):

The serve() method sets up a raw UDP socket. Notably, it includes cross-platform logic:

- **On Linux:** It attempts to bind directly to a physical interface (like eth0) using SO_BINDTODEVICE.
- **On Windows:** It binds to a specific IP or 127.0.0.1.
- **Broadcasts:** It enables SO_BROADCAST, which is essential because DHCP clients don't have an IP yet and communicate via broadcast MAC/IP addresses.

3

The DHCP State Machine (handle_packet):

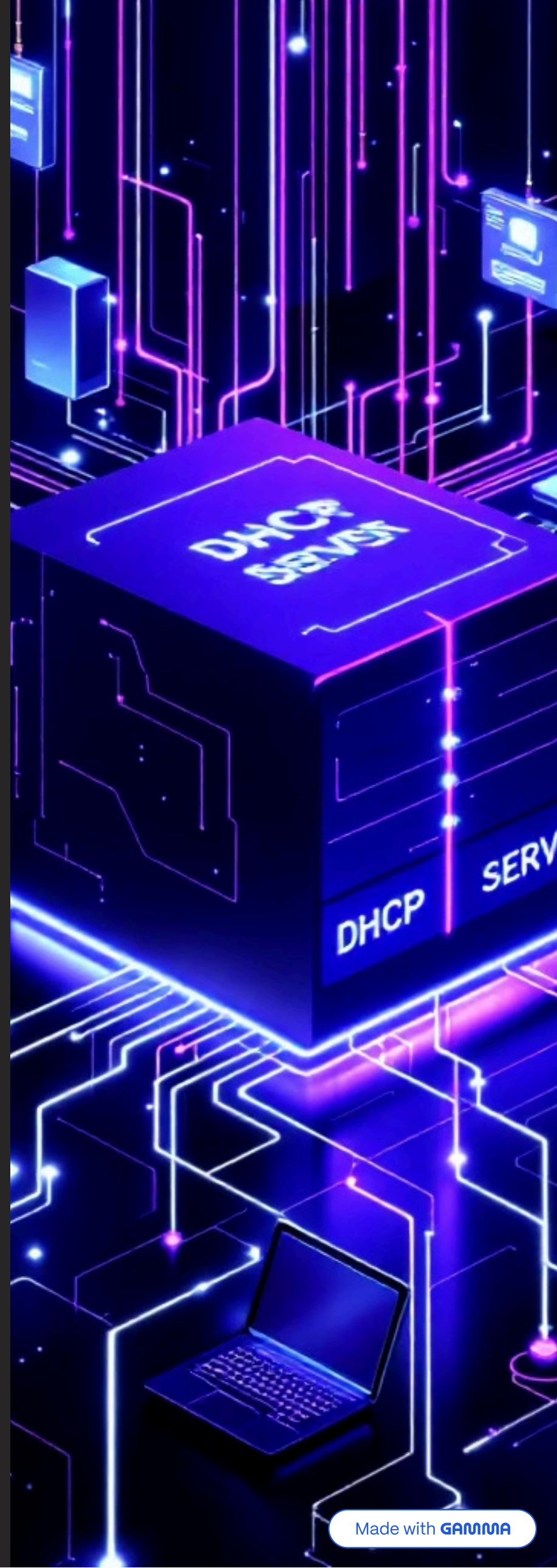
The server listens for incoming UDP packets and reacts based on the DHCP Message Type (Option 53):

- **DISCOVER:** When a client shouts "Is there a server out there?", the code picks the first available IP from the pool and returns an OFFER.
- **REQUEST:** When the client says "I'd like to use that IP you offered," the code validates the request and returns an ACK (Acknowledgment), finalizing the lease.

4

Utility Functions:

- **get_local_ip():** A clever trick that opens a dummy socket to 8.8.8.8 (Google DNS) just to see which local interface the OS prefers, then closes it immediately.
- **argparse:** Provides a CLI so you can specify the interface and base IP without hardcoding them.



DHCP/dhcp_service.py

Class Initialization

The DHCPService class initializes with a random transaction ID (xid) and retrieves the machine's MAC address using `uuid.getnode()`. The `_get_mac_binary()` method converts the node ID to a 6-byte binary representation.

```
import uuid
import socket
import time
import json
from dataclasses import dataclass, field
from typing import Dict, Any, Optional

# Assuming DHCPpacket, DHCPState, unpack_dhcp_packet,
pack_dhcp_packet,
# create_discover, create_request are imported from dhcp_protocol
# and get_local_ip from dhcp_server for client side logic
# (Actual imports would be more specific in a real project)

class DHCPService:
    def __init__(self):
        self.xid = int(time.time()) # Random transaction ID
        self.mac_address_binary = self._get_mac_binary()
        # Other initialization details as per project structure
```

The DORA Workflow (get_ip_and_dns)

This method executes the complete DHCP handshake:

- **Socket Setup:** Creates a UDP socket with `SO_REUSEADDR` and `SO_BROADCAST` enabled, binds to `CLIENT_PORT`, and sets a 5-second timeout.
- **DISCOVER:** Sends a broadcast DISCOVER packet to the server asking for available IP addresses.
- **OFFER:** Receives the server's OFFER response containing a proposed IP (yiaddr) and server IP (siaddr).
- **REQUEST:** Sends a REQUEST packet formally asking to lease the offered IP address.
- **ACK:** Receives the server's ACK confirmation with final lease parameters and DNS server information (Option 6).

```
def get_ip_and_dns(self) -> (Optional[str], Optional[str]):
    CLIENT_PORT = 68
    SERVER_PORT = 67
    BROADCAST_IP = '255.255.255.255'

    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
    sock.bind(('', CLIENT_PORT))
    sock.settimeout(5) # 5-second timeout for receiving responses

    try:
        # 1. DISCOVER
        discover_packet = create_discover(self.xid,
self.mac_address_binary)
        sock.sendto(pack_dhcp_packet(discover_packet), (BROADCAST_IP,
SERVER_PORT))

        # 2. OFFER
        data, server_address = sock.recvfrom(1024)
        offer_packet = unpack_dhcp_packet(data)

        # Basic validation
        if offer_packet.options.get(53) != DHCPState.OFFER:
            raise ValueError("Did not receive a DHCP OFFER")

        offered_ip = offer_packet.yiaddr
        server_ip = offer_packet.siaddr

        # 3. REQUEST
        request_packet = create_request(self.xid, self.mac_address_binary,
offered_ip, server_ip)
        sock.sendto(pack_dhcp_packet(request_packet), (BROADCAST_IP,
SERVER_PORT))

        # 4. ACK
        data, server_address = sock.recvfrom(1024)
        ack_packet = unpack_dhcp_packet(data)

        # Validate ACK packet
        if ack_packet.options.get(53) != DHCPState.ACK:
            raise ValueError("Did not receive a DHCP ACK")

        assigned_ip = ack_packet.yiaddr
        dns_server = ack_packet.options.get(6) # Option 6 for DNS Servers

        return assigned_ip, dns_server

    except socket.timeout:
        print("Socket timeout: No DHCP server response received.")
        return None, None
    except Exception as e:
        print(f"An error occurred during DHCP handshake: {e}")
        return None, None
    finally:
        sock.close()
```

Return Values

On successful completion, returns a tuple of (assigned_ip, dns_server). If a socket timeout occurs, returns (None, None).

Key Implementation Details

- Uses localhost (127.0.0.1) for testing purposes
- Extracts DNS server IP from Option 6 in the ACK packet
- Validates that the ACK packet has the correct DHCP Message Type (Option 53 == DHCPState.ACK)
- Handles socket timeouts gracefully with error messaging

DHCP/dhcp_protocol.py (JSON)

Core Functionality:

This section covers the fundamental operations of the JSON-based DHCP protocol implementation.

IP Address Management (The Pool):

The server manages a small subnet (defaulting to 10.100.102.x).

- **Allocation Logic:** It starts assigning IPs from .10 upwards.
- **State Persistence:** It uses a dictionary (self.pool) to map a transaction_id (xid) to a specific IP. This ensures that if the same client asks twice, it gets the same "reserved" address.

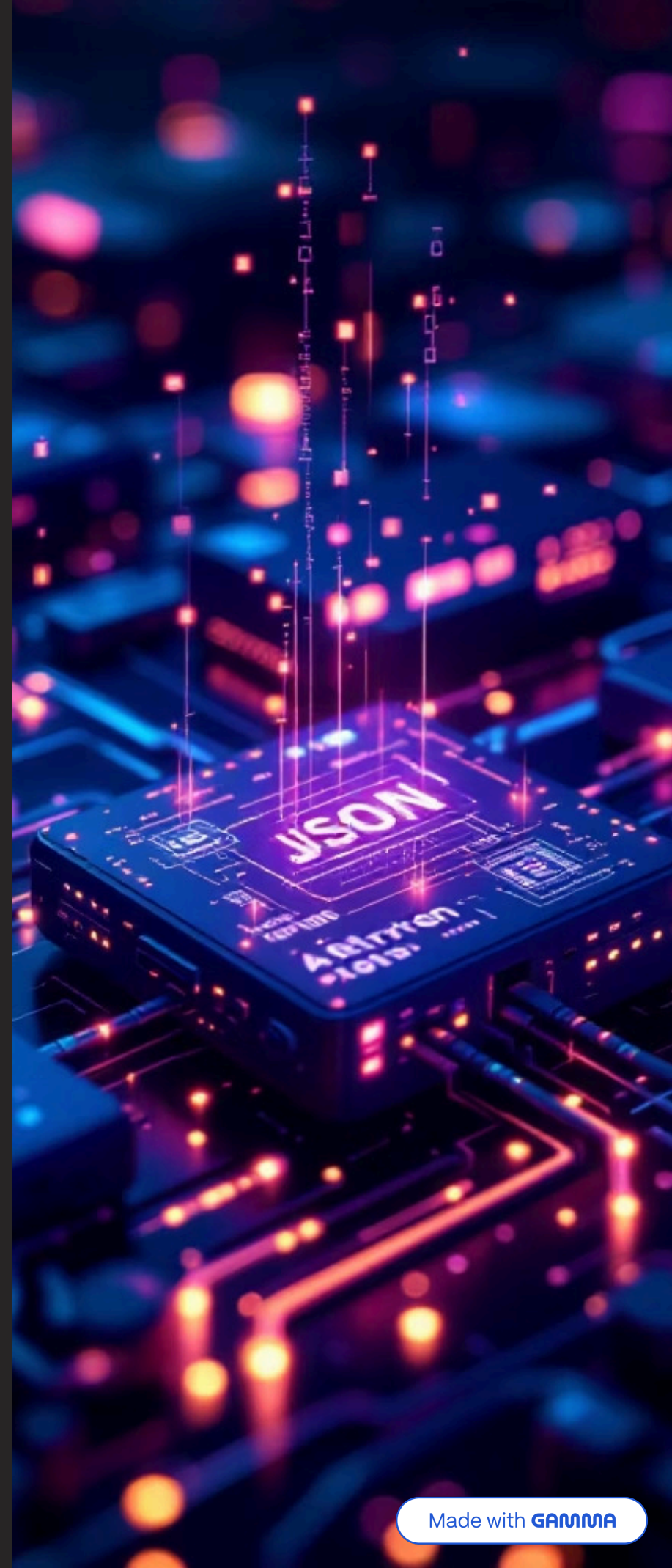
The Simulation Logic:

The handle method simplifies the 4-step DHCP handshake into a 2-step response trigger:

- **If DISCOVER:** It responds with an OFFER.
- **If REQUEST (or anything else):** It responds with an ACK.
- **Payload:** It sends back the assigned IP and a DNS server address (fetched from a local dhcp.json file or a default).

Network Transport:

- **Protocol:** It uses UDP (SOCK_DGRAM), which is correct for DHCP.
- **Port:** It listens on port 6767 (Standard DHCP uses 67, but that requires root/admin privileges, so this uses a non-privileged port).



DHCP/dhcp_server.py (JSON)

Core Functionality:

- **Network Range Definition:** It takes a 24-bit prefix (like 192.168.1) via `--ip-mask` to define the subnet the server will manage.
- **Pool Management:** The `--allocation` flag determines the size of the dynamic IP pool (how many "leases" the server can hand out).
- **DNS Configuration:** It allows for a static DNS server IP to be injected into the DHCP offers, which is crucial for directing client traffic to specific nameservers.
- **Execution Loop:** It initializes the `DHCPServer` object and enters an infinite loop (`server.serve()`) to listen for broadcast DISCOVER packets from clients on the network.

Logic Flow:

- **Argument Parsing:** Uses `argparse` to validate and ingest command-line inputs.
- **Object Initialization:** Passes the CLI parameters into the `DHCPServer` class.
- **Process Monitoring:** Prints a status header showing the active mask, pool size, and DNS.
- **Graceful Shutdown:** Includes a `try/except` block to catch `KeyboardInterrupt` (Ctrl+C), ensuring the server closes cleanly rather than throwing a messy stack trace.



DHCP/dhcp_service.py (JSON) - DORA Workflow

The Breakdown: The "DORA" Workflow

The code follows the four-step handshake required for a client to lease an IP address:

- **Discover:** The client broadcasts a message to the entire network looking for a server.
- **Offer:** A server responds with a potential IP address (offered_ip).
- **Request:** The client broadcasts back, saying "Yes, I would like to lease this specific IP."
- **Acknowledge (ACK):** The server confirms the lease and provides final details, like the DNS server.

Engineering Highlights

- **Socket Configuration:** It uses `socket.SOCK_DGRAM` (UDP) and sets `SO_BROADCAST`. This is crucial because, at the start of the process, the client has no IP address and doesn't know the server's IP, so it has to shout to everyone (255.255.255.255).
- **Transaction ID (xid):** A random integer is used to track the conversation. Since UDP is stateless and multiple clients might be broadcasting at once, the xid ensures the client only processes responses meant for its specific request.
- **Error Handling:** It includes a 5-second timeout and basic exception handling for JSON decoding, which is vital in networking where packets frequently go missing (packet loss).
- **Non-Standard Ports:** It uses ports 6767 and 6868. Standard DHCP uses 67 and 68, but using high ports allows this code to run without sudo/root privileges.



Client/RFC - Network Lifecycle

The Network Lifecycle (The main function): The main() function executes a classic network boot-up sequence. If this were a real computer, this is what happens the moment you plug in an Ethernet cable:

Phase 1: DHCP (Layer 3 Initializing)

- It calls the DHCPService we looked at earlier.
- **Goal:** Obtain a local IP address and the address of a DNS server.
- **Outcome:** If this fails, the script dies because the "device" has no identity on the network.

Phase 2: DNS (Name Resolution)

- Once it has a DNS server IP, it uses the DNSService to look up ftp.project_gay.com.
- **Goal:** Convert a human-readable hostname into a machine-readable IP address (server_ip).
- **Observation:** It passes the dns_ip (learned from DHCP) directly into the DNS resolver—exactly how a real OS works.

Phase 3: FTP (Application Layer)

- Finally, it initiates the FTPService.
- **Goal:** Transfer files to/from the resolved server_ip.
- **Note:** You'll notice it uses LOCAL_IP ("127.0.0.1") for the binding. In a production environment, it would likely use the assigned_ip from the DHCP phase, but it seems to be hardcoded for local testing here.



Client/JSON - Three-Phase Network Lifecycle

The Three-Phase Network Lifecycle: The main() function is a textbook example of the Dependency Chain in networking:



Phase 1: DHCP (Layer 3/4 Initialization)

Action: `dhcp.get_ip_and_dns()`

Logic: The client broadcasts to find a server. It receives a local IP address (`assigned_ip`) and the location of a DNS server (`dns_ip`).

Criticality: This is the "Identity" phase. Without this, the client doesn't know who it is or who to talk to for names.



Phase 2: DNS (Name Resolution)

Action: `dns.resolve(HOSTNAME)`

Logic: It takes the `dns_ip` it just learned from the DHCP phase and asks: "Where is `ftp.example.com`?"

Outcome: It gets back a specific IP (`server_ip`). This mirrors how your browser works every time you type a URL.



Phase 3: FTP (Application Layer)

Action: `ftp.run_file_transfer()`

Logic: Now that it knows where the server is (`server_ip`), it initiates the file transfer.

Observation: Notice it uses `LOCAL_IP` (`127.0.0.1`). This suggests the simulation is running locally on your machine, even though the logic mimics a remote connection.



